

TP n° 5

Transactions et Contrôle de Concurrence
Oracle APEX / SQLPLUS
Par DOUIRI Amjad

2026

Table des matières

1	Commande à exécuter au début de chaque session	2
2	Exercice n°1 : Atomicité d'une transaction	2
3	Exercice n°2 : Transactions concurrentes	4

1 Commande à exécuter au début de chaque session

```
1 SET AUTOCOMMIT OFF
```

Rappel : Ce TP nécessite d'ouvrir **deux sessions simultanées** (S1 et S2). En SQLPLUS : ouvrir deux terminaux. En Oracle APEX : ouvrir deux onglets SQL Commands.

2 Exercice n°1 : Atomicité d'une transaction

Question 1 – Créer la table transaction (Session S1)

```
1 CREATE TABLE transaction (  
2     idTransaction VARCHAR2(44),  
3     valTransaction NUMBER(10)  
4 );
```

Question 2 – Insertions, modification, suppression et ROLLBACK (Session S2)

```
1 -- Insertions  
2 INSERT INTO transaction VALUES ('TX001', 100);  
3 INSERT INTO transaction VALUES ('TX002', 200);  
4 INSERT INTO transaction VALUES ('TX003', 300);  
5  
6 -- Modification  
7 UPDATE transaction SET valTransaction = 999 WHERE idTransaction =  
8     'TX001';  
9  
10 -- Suppression  
11 DELETE FROM transaction WHERE idTransaction = 'TX002';  
12  
13 -- Verification avant annulation  
14 SELECT * FROM transaction;  
15  
16 -- Annulation de toutes les modifications  
17 ROLLBACK;  
18  
19 -- Verification apres rollback : la table est vide  
20 SELECT * FROM transaction;
```

Résultat attendu : Après ROLLBACK, la table est vide — toutes les modifications sont annulées.

Question 3 – Quitter sans COMMIT (Session S2)

```
1 INSERT INTO transaction VALUES ('TX010', 1000);
2 INSERT INTO transaction VALUES ('TX011', 2000);
3 -- Fermer sans COMMIT
4 quit;
```

```
1 -- Verifier dans S1
2 SELECT * FROM transaction;
3 -- Resultat : vide (ROLLBACK automatique au quit)
```

Résultat attendu : La table est **vide**. Quitter sqlplus sans COMMIT provoque un ROLLBACK automatique.

Question 4 – Fermeture brutale sans COMMIT (Session S1)

```
1 INSERT INTO transaction VALUES ('TX020', 5000);
2 -- Fermer brutalement (Ctrl+C ou tuer le processus)
3
4 -- Apres reconnexion :
5 SELECT * FROM transaction;
6 -- Resultat : vide (ROLLBACK automatique)
```

Résultat attendu : Les données **ne sont pas préservées**. Une déconnexion brutale déclenche un ROLLBACK automatique par Oracle (propriété d'**atomicité**).

Question 5 – Modification de structure + ROLLBACK

```
1 INSERT INTO transaction VALUES ('TX030', 100);
2
3 -- DDL provoque un COMMIT implicite automatique
4 ALTER TABLE transaction ADD (val2transaction NUMBER(10));
5
6 -- ROLLBACK sans effet
7 ROLLBACK;
8
9 -- Les lignes inserees AVANT le DDL sont permanentes
10 SELECT * FROM transaction;
11 -- TX030 est toujours la
```

Résultat attendu : Toute instruction DDL (CREATE, ALTER, DROP) provoque un **COMMIT implicite** sous Oracle. Le ROLLBACK n'a aucun effet sur les opérations déjà validées.

Question 6 – Conclusion : définitions

Concept	Définition
Session	Connexion d'un utilisateur à la base. Commence à la connexion, se termine à la déconnexion. Une session peut contenir plusieurs transactions en série.
Transaction	Suite d'opérations SQL formant une unité logique. Commence après le dernier COMMIT/ROLLBACK et se termine par COMMIT ou ROLLBACK.
COMMIT	Valide définitivement toutes les modifications de la transaction courante. Les données deviennent visibles par toutes les autres sessions.
ROLLBACK	Annule toutes les modifications depuis le dernier COMMIT. La base revient à son état précédent.

Propriétés ACID :

- **Atomicité** : Tout ou rien — soit toutes les opérations réussissent, soit aucune.
- **Cohérence** : La base passe d'un état cohérent à un autre état cohérent.
- **Isolation** : Chaque transaction s'exécute comme si elle était seule.
- **Durabilité** : Après un COMMIT, les données sont persistées même en cas de panne.

3 Exercice n°2 : Transactions concurrentes

Création du schéma

```
1 CREATE TABLE vol (  
2     idVol                VARCHAR2(44),  
3     capaciteVol          NUMBER(10),  
4     nbrPlacesReserveesVol NUMBER(10)  
5 );  
6  
7 CREATE TABLE client (  
8     idClient             VARCHAR2(44),  
9     prenomClient         VARCHAR2(11),  
10    nbrPlacesReserveesClient NUMBER(10)  
11 );  
12  
13 -- Donnees initiales  
14 INSERT INTO vol VALUES ('VOL001', 10, 0);  
15 INSERT INTO client VALUES ('C1', 'Alice', 0);  
16 INSERT INTO client VALUES ('C2', 'Bob', 0);  
17 COMMIT;
```

Isolation des transactions (READ COMMITTED)

Session S1 (T1) — réservation de 2 places pour Alice, sans COMMIT :

```

1 -- S1 : modifications non validees
2 UPDATE vol      SET nbrPlacesReserveesVol      =
  nbrPlacesReserveesVol + 2
3 WHERE idVol = 'VOL001';
4 UPDATE client SET nbrPlacesReserveesClient  =
  nbrPlacesReserveesClient + 2
5 WHERE idClient = 'C1';
6
7 -- S1 voit ses propres modifications
8 SELECT * FROM vol;      -- affiche 2
9 SELECT * FROM client;  -- Alice = 2

```

Session S2 (T2) — lecture pendant que T1 n'a pas commité :

```

1 -- S2 ne voit PAS les modifications de T1
2 SELECT * FROM vol;      -- affiche 0
3 SELECT * FROM client;  -- Alice = 0

```

Observation : Les modifications de T1 sont **invisibles** pour T2 tant que T1 n'a pas fait de COMMIT — c'est la propriété d'**isolation**.

ROLLBACK de T1

```

1 -- S1 : annuler
2 ROLLBACK;
3
4 -- Verifications
5 SELECT * FROM vol;      -- 0 dans les deux sessions
6 SELECT * FROM client;  -- 0 dans les deux sessions

```

Problème des mises à jour perdues (READ COMMITTED)

Ordre	Session S1 (T1)	Session S2 (T2)
1	SELECT vol → 0	SELECT vol → 0
2		
3	UPDATE vol SET vol = 0+2 UPDATE client C1 = 0+2 COMMIT;	
4		UPDATE vol SET vol = 0+3 UPDATE client C2 = 0+3 COMMIT;
Résultat : vol = 3 (attendu : 5) → INCOHÉRENCE		

```

1 -- Etape 1 : S1 lit
2 SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'VOL001';
3 -- Etape 2 : S2 lit

```

```

4 SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'VOL001';
5 -- Etape 3 : S1 ecrit + COMMIT
6 UPDATE vol SET nbrPlacesReserveesVol = 0 + 2 WHERE idVol
   = 'VOL001';
7 UPDATE client SET nbrPlacesReserveesClient = 0 + 2 WHERE
   idClient = 'C1';
8 COMMIT;
9 -- Etape 4 : S2 ecrit + COMMIT (ecrase T1 !)
10 UPDATE vol SET nbrPlacesReserveesVol = 0 + 3 WHERE idVol
   = 'VOL001';
11 UPDATE client SET nbrPlacesReserveesClient = 0 + 3 WHERE
   idClient = 'C2';
12 COMMIT;
13 -- Verification
14 SELECT * FROM vol; -- 3 au lieu de 5 : MISE A JOUR PERDUE

```

Isolation complète (SERIALIZABLE)

```

1 -- Reinitialisation
2 UPDATE vol SET nbrPlacesReserveesVol = 0;
3 UPDATE client SET nbrPlacesReserveesClient = 0;
4 COMMIT;
5
6 -- Activer SERIALIZABLE dans S1 ET S2
7 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
8
9 -- Meme sequence que precedemment...
10 -- T2 sera rejetee par Oracle avec :
11 -- ORA-08177: can't serialize access for this transaction
12 ROLLBACK;

```

Résultat : En mode SERIALIZABLE, Oracle détecte le conflit et **rejette T2**. La cohérence est garantie.

Séquence concurrente $r_1(d)$ $w_2(d)$ $w_2(d')$ C_2 $w_1(d')$ C_1

```

1 -- S1 : r1(d)
2 SELECT nbrPlacesReserveesVol FROM vol WHERE idVol = 'VOL001';
3
4 -- S2 : w2(d)
5 UPDATE vol SET nbrPlacesReserveesVol = 5 WHERE idVol = 'VOL001';
6 -- S2 : w2(d')
7 UPDATE vol SET nbrPlacesReserveesVol = 7 WHERE idVol = 'VOL001';
8 -- S2 : C2 (libere le verrou)
9 COMMIT;
10
11 -- S1 : w1(d') -- possible car T2 a commite
12 UPDATE vol SET nbrPlacesReserveesVol = 9 WHERE idVol = 'VOL001';
13 -- S1 : C1
14 COMMIT;

```